CONTRIBUTION-PROFILE-AWARE MULTIMODAL LEARNING

Complete architecture, mathematical specification
and reproducible implementation plan

Version 1.2 · 13 October 2026

Scope. A dataset-agnostic framework for long-tailed classification with paired modalities. Example domains include emotion recognition, audio-visual event recognition and multimodal sentiment analysis.
Pipeline. data preparation → representation learning → contribution estimation → within-class grouping → class/group-balanced learning → CI–MI optimization → evaluation.

Document map

1 Problem definition and notation 1
2 End-to-end architecture 2
3 Data preparation and leakage control 3
4 Multimodal representation learning 4
5 Modality-removal contribution signatures 5
6 Target-versus-current alignment 6
7 Within-class contribution grouping 7
8 Class/group-balanced classification 8
9 Fixed-weight optimization and MOO 9
10 Algorithms and pseudocode 10
11 Datasets across application domains 11
12 Experimental setup and evaluation 13
13 Reproducibility and implementation checklist 14
14 References and resources 15

Notation convention. Defaults are reproducible starting points, not universal constants. Dataset adapters provide data in a common schema; the core model and optimization code remain dataset-independent.

1 Problem definition and notation

We consider supervised classification from two paired modalities A and V. These may be audio and video, but the formulation is not tied to them. Example tasks include emotion, event, activity, sentiment, medical or sensor-based classification whenever paired inputs and labels are available.

$$\mathcal{D} = \{(x_i^A, x_i^V, y_i, s_i)\}_{i=1}^N, \qquad y_i \in \{1, \dots, K\}. \tag{1.1}$$

Here x_i^A and x_i^V are paired observations, y_i is the class, s_i is optional source metadata (speaker, actor, recording session, device or video ID) and K is the number of classes. The class counts and class frequencies are

$$n_c = \sum_{i=1}^N \mathbf{1}[y_i = c], \qquad \pi_c = \frac{n_c}{N}. \tag{1.2}$$

- **Class imbalance (CI):** the class counts n_c differ substantially.
- **Modality imbalance (MI):** the modalities differ in predictive usefulness, and that usefulness may vary across classes or examples.

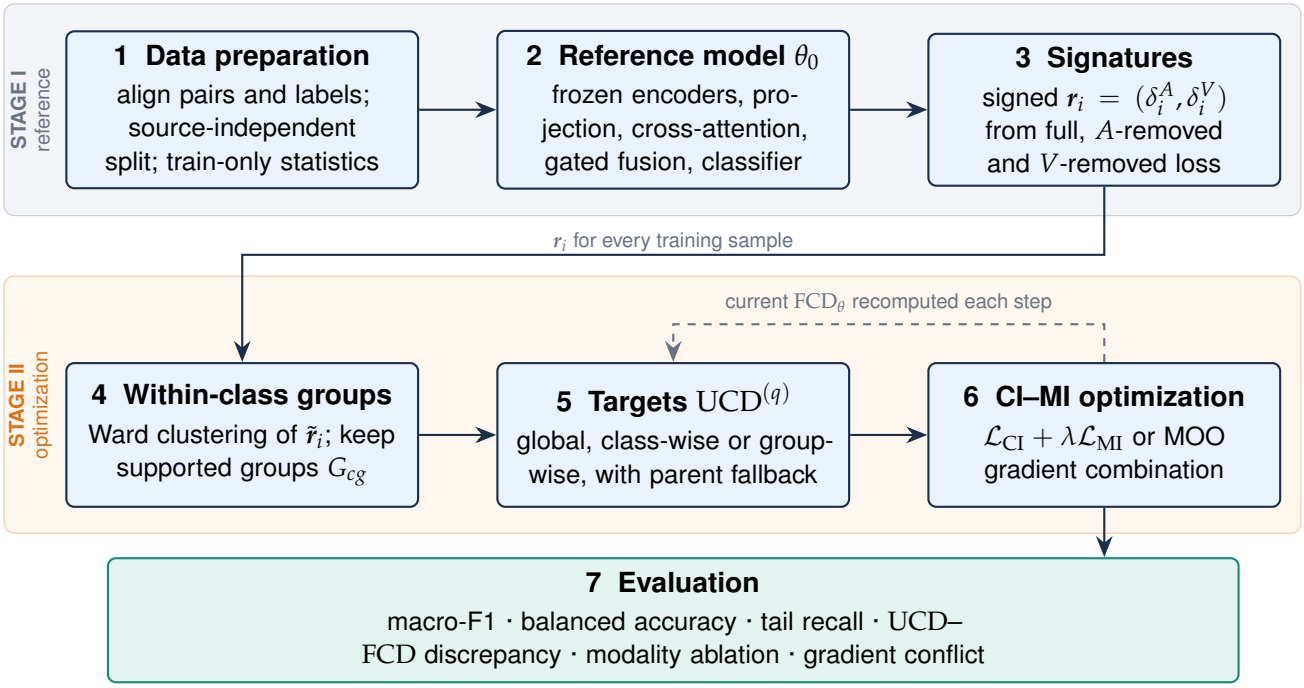


Figure 1. End-to-end pipeline. Stage I fixes the reference checkpoint θ_0 and measures contributions; Stage II uses the fixed targets and groups to optimize CI and MI jointly.

Equal contribution is *not* assumed to be optimal.

Symbol	Definition
$E_A,\ E_V$	Audio and video encoders.
$z_i^A,\ z_i^V$	Projected modality representations.
z_i^F	Fused representation.
θ	Trainable projection, fusion and classifier parameters; encoder parameters only if explicitly unfrozen.
ℓ_i^{AV}	Cross-entropy loss using both modalities.
$\ell_i^{-A},\ \ell_i^{-V}$	Loss when A or V is removed through a trained missing-modality path.
r_i	Signed sample signature (δ_i^A, δ_i^V).
UCD	Target contribution distribution, estimated from risk-based modality analysis.
FCD_θ	Current (factual) contribution distribution, estimated from the current model.
$G_c,\ G_{cg}$	Retained contribution groups within class c, and group g of class c.

2 End-to-end architecture

The pipeline separates representation learning, contribution measurement, grouping and optimization. Targets and group assignments are estimated from training data and then used in a second optimization stage.

Component	Reference stage	Optimization stage
Pretrained encoders	Frozen initially; cache features.	Frozen initially; optional partial unfreezing is a separate experiment.
Projection / attention / fusion	Train.	Train.
Classifier	Train.	Train.
Reference checkpoint	Created at end of reference training.	Fixed for target/profile estimation.
Target UCD and groups	Not yet used.	Estimated on training data; fixed in the main experiment.
Current FCD	Not required for baseline training.	Recomputed as the current model changes.

> Use the *same* classifier for full-modality and modality-removal evaluation. Separate classifiers for each condition would confound contribution measurement.

3 Data preparation and leakage control

Each dataset adapter exports the same record format. The core training code must not depend on dataset-specific filenames.

```
sample_id | audio_path_or_features | video_path_or_features | label_id
source_id | split_id | time_start | time_end
```

Preparation procedure

1. Read official annotations and create a sample manifest; verify labels and both modalities.
2. Align the modalities to the same utterance, event or temporal interval. Record the exact windowing rule.
3. Flag corrupted, missing, duplicate or invalid examples and save an exclusion log.
4. Split *before* fitting models, scalers, target estimators or clusters. Prefer speaker/actor/source-independent splits.
5. Fit all normalization statistics and profile-grouping rules on training data only.
6. Use validation for model selection and leave the test set untouched until final evaluation.

Controlled long-tailed protocol

Report natural class counts first. If the dataset is not sufficiently long-tailed, create controlled training subsets by downsampling *training data only*; keep validation and test distributions fixed. With classes ranked $c = 1, \ldots, K$ by size and imbalance ratio $\rho = n_{\max}/n_{\min}$, a geometric schedule is

$$n_c = \left\lfloor n_{\max} \rho^{-\frac{c-1}{K-1}} \right\rfloor, \tag{3.1}$$

limited by the number of available samples. Report the exact per-class counts and the sampling seed.

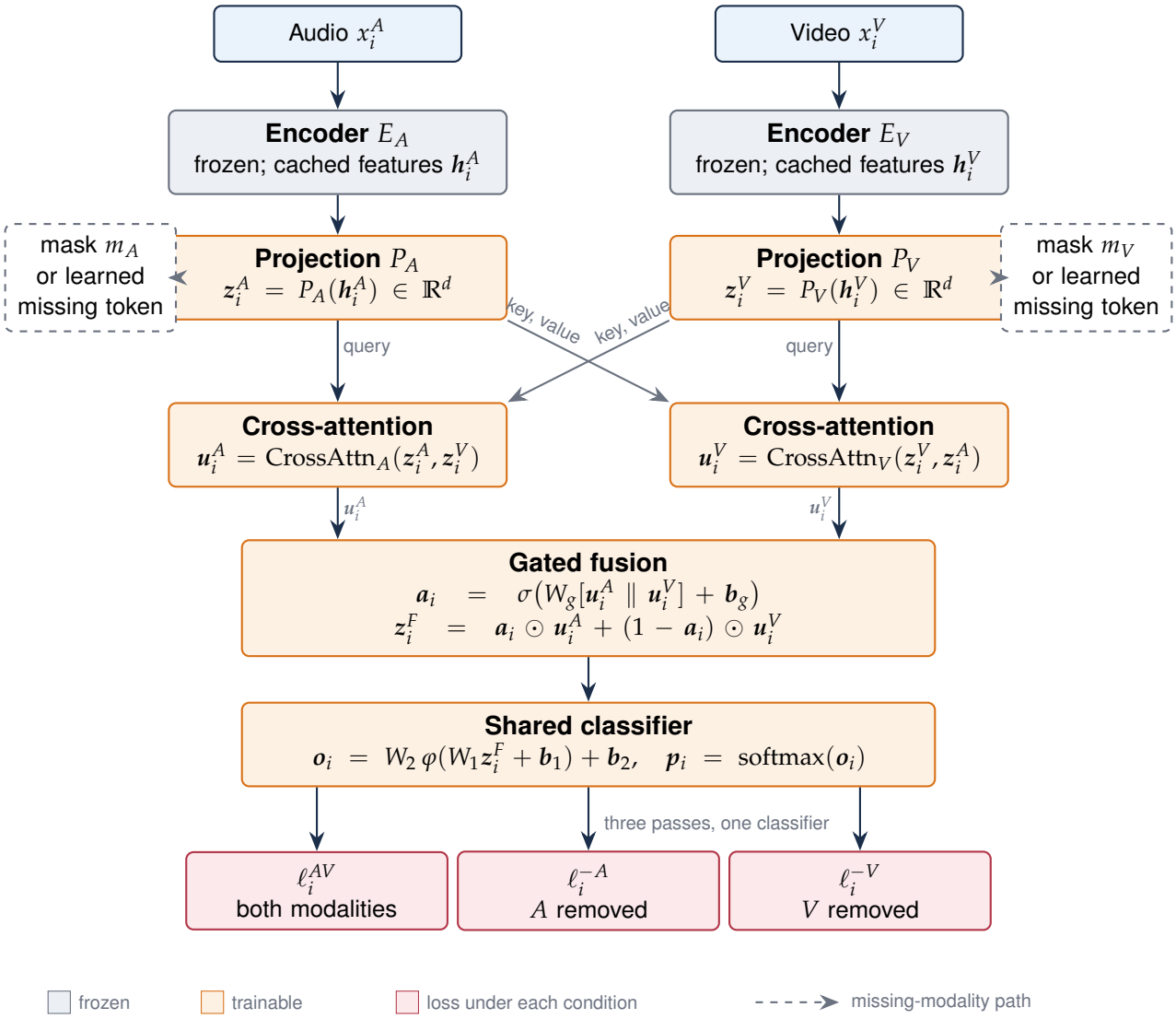


Figure 2. Model architecture. Pretrained encoders stay frozen; projections, attention, fusion and classifier are trained. The missing-modality path replaces a removed stream before attention.

Leakage rule. Contribution estimation may use out-of-fold predictions within the training split or a dedicated training-only estimation subset. Test examples and labels must never be used to estimate targets, fit groups, tune thresholds or choose checkpoints.

4 Multimodal representation learning

Figure 2 shows the model. Pretrained encoders produce modality features, projections map them to a common space, bidirectional cross-attention lets each modality condition on the other, and a learned gate fuses the result. One shared classifier produces the full-modality loss and both modality-removal losses.

4.1 Encoders and projections

$$h_i^A = E_A(x_i^A), \quad h_i^V = E_V(x_i^V), \qquad z_i^A = P_A(h_i^A) \in \mathbb{R}^d, \quad z_i^V = P_V(h_i^V) \in \mathbb{R}^d. \tag{4.1}$$

4.2 Cross-modal attention and gated fusion

Bidirectional cross-attention lets each modality condition on the other; a learned gate then fuses the two streams:

$$u_i^A = \mathrm{CrossAttn}_A(z_i^A, z_i^V), \qquad u_i^V = \mathrm{CrossAttn}_V(z_i^V, z_i^A), \tag{4.2}$$

$$a_i = \sigma\big(W_g[u_i^A \,\|\, u_i^V] + b_g\big), \qquad z_i^F = a_i \odot u_i^A + (1 - a_i) \odot u_i^V. \tag{4.3}$$

Here $\|$ is concatenation and $\odot$ is the elementwise product. For temporal features, document the pooling or sequence-aggregation operation and use it consistently.

4.3 Classifier and reference objective

$$o_i = W_2\,\varphi(W_1 z_i^F + b_1) + b_2, \qquad p_i = \mathrm{softmax}(o_i), \tag{4.4}$$

$$\ell_i^{AV}(\theta) = -\log p_\theta(y_i \mid x_i^A, x_i^V), \qquad \mathcal{L}_{\mathrm{ref}}(\theta) = \frac{1}{N_{\mathrm{train}}} \sum_i \ell_i^{AV}(\theta). \tag{4.5}$$

Train with natural sampling and ordinary cross-entropy to obtain the reference model θ_0. On a 6 GB GPU, start with pretrained frozen encoders and cached features; train projections, fusion and classifier. Save the checkpoint and all preprocessing settings.

4.4 Valid missing-modality path

Train a modality-presence mask or a learned missing token and expose the model to modality dropout during reference training. At inference, remove A or V through that trained path. Do not assume an arbitrary zero vector is a valid missing-modality representation unless the model was trained and validated that way.

5 Modality-removal contribution signatures

Evaluate the reference model under three conditions: both modalities, A removed and V removed. Use the same ground-truth label and the shared classifier; no ablation-specific classifier is trained.

$$\delta_i^A = \ell_i^{-A}(\theta_0) - \ell_i^{AV}(\theta_0), \qquad \delta_i^V = \ell_i^{-V}(\theta_0) - \ell_i^{AV}(\theta_0), \qquad r_i = \begin{bmatrix} \delta_i^A \\ \delta_i^V \end{bmatrix}. \tag{5.1}$$

Positive δ means removal *increased* loss; negative δ means removal *decreased* loss. Retain the signed values, since they can reveal modality interference or sample-specific behaviour.

> r_i is a signed vector, not a probability distribution, so it must not be passed directly to a KL divergence. Use it for within-class grouping and diagnostics.

Reliability procedure

- Prefer out-of-fold reference predictions or a dedicated training-only estimation subset.
- Use the same trained missing-modality path for all samples and experiments.
- Report per-class distributions of δ^A and δ^V, including negative values.

- Bootstrap the training estimation set to inspect uncertainty in class and group summaries.
- If a group estimate is too small or unstable, fall back to the class-level target; if that is unreliable, use the global target.

> Keep three quantities separate: the signed loss signature $\boldsymbol{r}_i$, the target distribution UCD and the current distribution FCD_θ. They measure related but non-identical things.

6 Target-versus-current contribution alignment

This stage follows the Utopia-inspired principle of estimating a desired contribution distribution (UCD) and aligning the current (factual) contribution distribution (FCD) to it. The architecture evaluates global, class-wise and within-class group-wise target scopes q.

6.1 Target contribution distribution (UCD)

For scope q (global, class c or group (c, g)), define a nonnegative target distribution over the two modalities, $\boldsymbol{u}^{*(q)} = [u_A^{*(q)}, u_V^{*(q)}]$ with $u_m^* \geq 0$ and $\sum_m u_m^* = 1$. A transparent risk-based implementation uses the increase in mean classification risk when modality m is removed:

$$d_m^{(q)} = \max\Big(0,\ R^{(q)}(\text{without } m) - R^{(q)}(\text{full})\Big), \tag{6.1}$$

$$u_m^{*(q)} = \frac{d_m^{(q)} + \varepsilon}{\sum_{j\in\{A,V\}} \big(d_j^{(q)} + \varepsilon\big)}. \tag{6.2}$$

$R^{(q)}$ is the mean classification risk over the samples in scope q and ε is a small constant.

> This is a reproducible risk-based *instantiation* of the target principle. To reproduce Utopia exactly, implement its published UCD estimator rather than assuming this simplified formula is identical to the paper.

6.2 Current (factual) contribution distribution (FCD)

For current parameters θ, estimate each modality's information contribution $f_m^{(q)}(\theta)$ to the fused representation with a documented nonnegative estimator, then normalize:

$$v_m^{(q)}(\theta) = \frac{f_m^{(q)}(\theta) + \varepsilon}{\sum_{j\in\{A,V\}} \big(f_j^{(q)}(\theta) + \varepsilon\big)}. \tag{6.3}$$

The Utopia-faithful path uses its mutual-information-based FCD estimator and the prescribed approximation. If an ablation-based current estimator is used instead, label and evaluate it as a separate variant.

6.3 Alignment loss

$$\mathcal{L}_{\mathrm{MI}}^{(q)}(\theta) = D_{\mathrm{KL}}\big(\boldsymbol{u}^{*(q)} \,\|\, \boldsymbol{v}_\theta^{(q)}\big) = \sum_{m\in\{A,V\}} u_m^{*(q)} \log \frac{u_m^{*(q)} + \varepsilon}{v_m^{(q)}(\theta) + \varepsilon}. \tag{6.4}$$

Use the target-to-current KL direction as written; clip and renormalize both distributions before the calculation, and report ε and the exact UCD/FCD estimators.

6.4 Granularity and fallback

If an estimate at a scope is unreliable, the target falls back to its parent scope, down to the global root target.

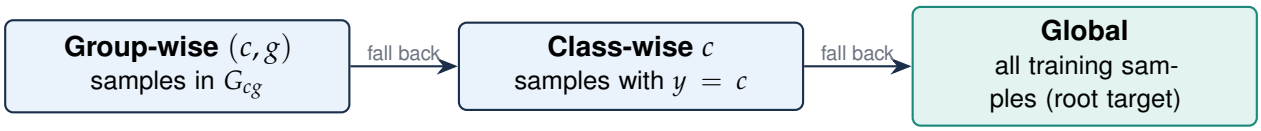


Figure 3. Target scopes, the sample set used to estimate each target, and the fallback chain.

7 Within-class contribution grouping

Group samples inside each original class using the signed signatures $\boldsymbol{r}_i$.

> This is inspired by within-class subdivision, but it is *not* the original SBCL method: the clustering signal is modality-contribution behaviour rather than representation-space distance.

7.1 Scaling

$$\tilde{r}_{i,m} = \frac{r_{i,m} - \mu_m}{\sigma_m + \varepsilon}, \qquad m \in \{A, V\}. \tag{7.1}$$

Compute μ and σ on the training estimation set only. Use robust scaling if outliers dominate, and keep the scaling method fixed across compared methods.

7.2 Reproducible clustering algorithm

1. For each class c, collect the standardized signatures $\tilde{\boldsymbol{r}}_i$ of samples with $y_i = c$.
2. If class support is below the predeclared minimum or profile dispersion is negligible, retain one group.
3. Otherwise run agglomerative clustering on the two-dimensional signatures (Euclidean distance, Ward linkage).
4. Select the group count from a small predeclared range, such as 1–3 groups per class, subject to minimum support and training-only stability or silhouette checks.
5. Save group IDs, counts, centroids, covariance, mean signed signature and target distribution.
6. Freeze group assignments for the main optimization experiment; dynamic reclustering is a separate experiment.

7.3 Minimum support

Set $m_{\min}$ before the main experiment. A pilot starting point is

$$m_{\min} = \max\big(20,\ \lceil 0.02\, n_c \rceil\big), \tag{7.2}$$

adjusted for dataset scale and then fixed. Merge undersized groups with their nearest centroid or retain a single parent group. Log every merge and fallback. Do not force all classes to have the same group count.

7.4 Group statistics

$$\bar{r}_{cg} = \frac{1}{|G_{cg}|} \sum_{i \in G_{cg}} r_i, \qquad \Sigma_{cg} = \mathrm{Cov}\big(\{r_i : i \in G_{cg}\}\big). \tag{7.3}$$

These statistics diagnose profile similarity and uncertainty. The group mean signature is *not* substituted directly for UCD; target estimation remains defined in Section 6.

8 Class/group-balanced classification

8.1 Class-balanced loss

Each original class contributes equal total weight. This is the primary CI baseline.

$$\mathcal{L}_{\text{CI-class}}(\theta) = \frac{1}{K} \sum_{c=1}^{K} \frac{1}{n_c} \sum_{i:\, y_i = c} \ell_i^{AV}(\theta). \tag{8.1}$$

8.2 Hierarchical class/group-balanced loss

Each class contributes $1/K$; each retained group within a class contributes equally; samples are averaged within each group. The contribution groups therefore affect CI directly, not only the MI targets.

$$\mathcal{L}_{\text{CI-group}}(\theta) = \frac{1}{K} \sum_{c=1}^{K} \left[\frac{1}{|G_c|} \sum_{g \in G_c} \frac{1}{|G_{cg}|} \sum_{i \in G_{cg}} \ell_i^{AV}(\theta) \right]. \tag{8.2}$$

8.3 Mini-batch implementation

Use a class-then-group stratified batch sampler, or per-example weights

$$w_i \propto \frac{1}{K\,|G_c|\,|G_{cg}|}, \qquad i \in G_{cg}. \tag{8.3}$$

Normalize weights within each batch or use an unbiased stratified sampler. Do not combine inverse-frequency weights with an already balanced sampler without checking for double correction.

8.4 Stabilization

- Enforce minimum group support and merge tiny groups according to the fixed rule.
- Compare class-balanced and group-balanced loss independently.
- Use the same optimizer, weight decay and gradient-clipping settings across compared methods.
- Do not add difficulty weights or group-DRO penalties to the core method; evaluate them only as separately named extensions.

9 Fixed-weight optimization and MOO

The two objectives are class/group-balanced classification and contribution alignment:

$$\boldsymbol{F}(\theta) = \left[\mathcal{L}_{\mathrm{CI}}(\theta),\ \mathcal{L}_{\mathrm{MI}}^{(q)}(\theta)\right]. \tag{9.1}$$

9.1 Fixed-weight baseline

$$\mathcal{L}_{\mathrm{fixed}}(\theta) = \mathcal{L}_{\mathrm{CI}}(\theta) + \lambda\, \mathcal{L}_{\mathrm{MI}}^{(q)}(\theta). \tag{9.2}$$

Tune λ on a validation-only log-spaced grid (starting example: 0.01, 0.1, 1, 10) after inspecting objective scales. Report the grid and the chosen value.

9.2 Genuine multi-objective optimization

With $g_{\mathrm{CI}} = \nabla_\theta \mathcal{L}_{\mathrm{CI}}$ and $g_{\mathrm{MI}} = \nabla_\theta \mathcal{L}_{\mathrm{MI}}$, a two-objective gradient-combination method chooses a step that considers both gradients. One transparent baseline is the minimum-norm convex combination:

$$g^* = \alpha^* g_{\mathrm{CI}} + (1-\alpha^*) g_{\mathrm{MI}}, \qquad \alpha^* = \operatorname*{arg\,min}_{\alpha\in[0,1]} \left\| \alpha\, g_{\mathrm{CI}} + (1-\alpha)\, g_{\mathrm{MI}} \right\|_2^2. \tag{9.3}$$

For two gradients this has the closed form

$$\alpha^* = \mathrm{clip}_{[0,1]}\left(\frac{(g_{\mathrm{MI}} - g_{\mathrm{CI}})^\top g_{\mathrm{MI}}}{\left\| g_{\mathrm{CI}} - g_{\mathrm{MI}} \right\|_2^2} \right), \tag{9.4}$$

which covers both endpoints and the clipped stationary solution (take $\alpha^* = \frac{1}{2}$ if the two gradients coincide). A vetted MGDA implementation is also acceptable. Record the exact algorithm, trainable parameter set and update rule. This differs from a fixed λ because the combination is recomputed from the current gradients.

9.3 Gradient conflict

$$\cos(g_{\mathrm{CI}}, g_{\mathrm{MI}}) = \frac{g_{\mathrm{CI}} \cdot g_{\mathrm{MI}}}{\|g_{\mathrm{CI}}\|_2 \, \|g_{\mathrm{MI}}\|_2 + \varepsilon}. \tag{9.5}$$

Log the cosine similarity and the proportion of steps with negative cosine. If the objectives rarely conflict, MOO may not improve over fixed-weight training; report the result rather than assuming MOO is superior.

9.4 Second-stage update loop

1. Initialize from the reference checkpoint θ_0; freeze encoders in the primary experiments.
2. Compute the full-modality classification loss and the selected CI objective.
3. Compute the current FCD using the documented estimator and the valid modality-removal (mask) path.
4. Calculate $\mathcal{L}_{\mathrm{MI}}$ for the selected granularity.
5. Update with either the fixed weighted sum or the chosen MOO algorithm.
6. Validate macro-F1, balanced accuracy, per-class recall and contribution discrepancy.
7. Select a checkpoint with a predeclared validation rule; test only after selection.

If the FCD estimator is non-differentiable, define a differentiable approximation or an alternating update procedure. Do not imply gradients flow through a quantity when the implementation does not support them.

10 Algorithms and pseudocode

Algorithm 1 · Estimate targets and groups

Input: training data, reference model θ_0, target scope q, minimum group support $m_{\min}$.
Output: UCD targets, current/reference contribution estimates, fixed group IDs and group statistics.

1. Train the reference model with natural-sampling cross-entropy.
2. For each training sample, evaluate the full, A-removed and V-removed losses.
3. Store the signed signature $\boldsymbol{r}_i = (\delta_i^A, \delta_i^V)$.
4. Estimate the global UCD and the reference FCD with documented estimators.
5. Estimate class and group targets with support-aware parent fallback.
6. Cluster signatures within each class only when the support and stability checks pass.
7. Save targets, group IDs, counts, scaler parameters, seeds and diagnostics.

Algorithm 2 · Optimize CI and MI

Input: θ_0, training split, groups, UCD targets, mode $\in \{\text{fixed-weight}, \text{MOO}\}$.
Output: selected model θ^*.

1. Initialize $\theta \leftarrow \theta_0$; freeze encoders for primary runs.
2. For each batch, compute $\mathcal{L}_{\text{CI}}$.
3. Estimate the current FCD and compute $\mathcal{L}_{\text{MI}}$.
4. *Fixed-weight mode:* update using $\nabla(\mathcal{L}_{\text{CI}} + \lambda\mathcal{L}_{\text{MI}})$.
5. *MOO mode:* compute each gradient and apply the chosen gradient-combination algorithm.
6. Log objective values, gradient norms and cosine, class/group metrics and learning rate.
7. Select θ^* by the validation rule; evaluate once on the test set.

Training pseudocode

```
for batch in train_loader:
    logits   = model(audio, video, modality_mask=(1, 1))
    loss_ci  = class_or_group_balanced_ce(logits, labels, group_ids)
    fcd      = estimate_fcd(model, batch, target_granularity,
                            trained_missing_modality_path)
    loss_mi  = kl_divergence(ucd_targets, fcd)

    if mode == "fixed_weight":
        loss = loss_ci + lambda_mi * loss_mi
        optimizer.zero_grad()
```

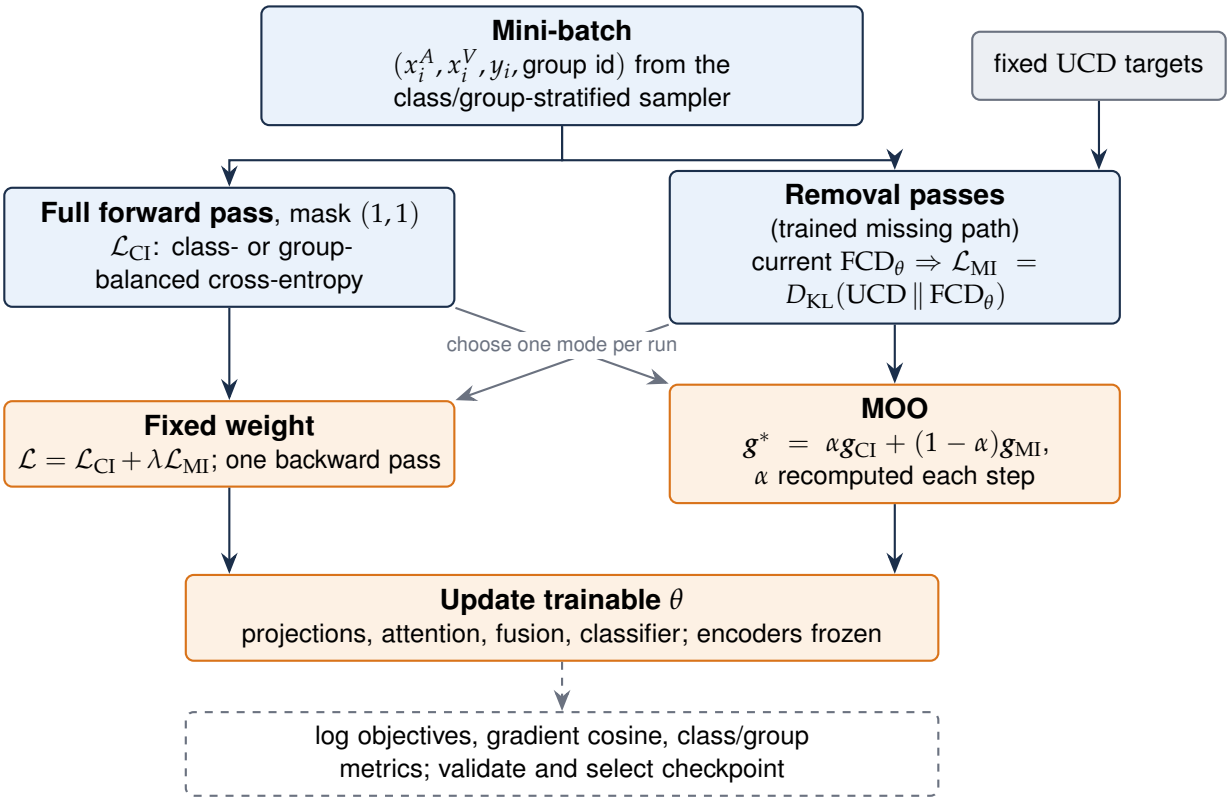


Figure 4. One optimization step of Algorithm 2. The fixed-weight and MOO modes use the same trainable parameter set.

```
        loss.backward()
        optimizer.step()
    else:  # MOO
        g_ci = compute_gradient(loss_ci, trainable_parameters)
        g_mi = compute_gradient(loss_mi, trainable_parameters)
        g    = moo_combine(g_ci, g_mi)
        optimizer.zero_grad()
        assign_gradients(trainable_parameters, g)
        optimizer.step()
```

Implementation contract. The code must specify how the FCD estimator is differentiated, how modality-removal passes are batched, and how gradients are collected without stepping twice. These are part of the specification, not optional undocumented details.

11 Datasets across application domains

Use one dataset to validate the end-to-end mechanics, then reproduce the protocol in a second domain. All adapters must export the same manifest schema and preserve the source-defined split when possible.

Dataset / domain	Task and modalities	Implementation notes
RAVDESS affective computing	Audio-video speech/song with emotion labels and actor metadata.	Start with paired speech clips. Parse the filename fields (modality, channel, emotion, intensity, statement, repetition, actor). Split by actor where possible; avoid mixing speech and song unless explicitly planned.
AVE audio-visual event recognition	10-second video clips, 28 event categories and annotated audio-visual event intervals.	Use official train/validation/test IDs. Align both streams to the event interval or use a documented fixed window. A video may contain several event instances; keep source video IDs to prevent leakage.
CMU-MOSI / MOSEI sentiment and affect	Opinion/sentence video segments with acoustic, visual and text information; sentiment labels and, for MOSEI, emotion annotations.	For the two-modality initial implementation, use acoustic and visual streams only. Retain official segment labels. Text can be added later as a third modality by generalizing all modality-indexed equations.

Resources

- RAVDESS: official Zenodo record
- AVE: official project repository
- CMU-MOSI: CMU MultiComp page
- CMU-MOSEI: CMU MultiComp page
- CMU Multimodal SDK: official GitHub repository

Dataset-specific implementation sequence

- **RAVDESS:** verify alignment, label parsing, speaker-independent splitting, profile estimates and missing-modality paths.
- **AVE:** verify event-window alignment, multi-instance/source-video handling and transfer beyond affective data.
- **MOSI/MOSEI:** verify segment-level alignment and speaker-independent evaluation; use only audio and video for the initial two-modality protocol.

Controlled imbalance

For each dataset, report the natural class distribution. If required, downsample only training examples to predeclared imbalance ratios such as $\rho \in \{10, 20, 50\}$, where class counts permit. Publish the per-class counts, split IDs and random seeds. Do not claim a dataset is naturally long-tailed without reporting its actual distribution.

12 Experimental setup and evaluation

ID	CI objective	MI objective / optimizer	Question
E0	Natural-sampling CE	None	Reference baseline.
E1	Class-balanced CE	None	Does conventional CI handling help?
E2	Hierarchical class/group CE	None	Does profile grouping help CI?
E3	Class-balanced CE	UCD–FCD alignment, fixed λ	Does MI alignment help without group-balanced CI?
E4	Hierarchical class/group CE	UCD–FCD alignment, fixed λ	Does group-based CI + MI help together?
E5	Hierarchical class/group CE	UCD–FCD alignment with MOO	Does genuine MOO improve over fixed-weight training?

Run E3–E5 at global, class-wise and group-wise target granularity. Keep the backbone, splits, training budget and target estimator fixed when testing granularity.

Metrics

Metric	What to report
Macro-F1	Per-class F1 averaged equally across classes.
Balanced accuracy	Mean per-class recall for single-label classification.
Per-class precision / recall / F1	Every class; optionally summarize head, medium and tail classes by training counts.
Top-1 accuracy	Secondary metric; it can hide tail-class failures.
UCD–FCD discrepancy	Target/current distribution divergence by scope and granularity.
Modality ablation	$A+V$, A-only and V-only results using the trained missing-modality path.
Gradient conflict	MOO gradient cosine distribution and fraction of negative-cosine steps.
Compute	GPU, peak memory, epochs, runtime, parameter count.

Protocol

- Use at least 3 random seeds for pilot runs and 5 when feasible; report mean $\pm$ standard deviation.
- Use identical splits and paired seeds for method comparisons.
- Select checkpoints and hyperparameters using validation only.
- Report class counts and sample counts behind every metric.
- Use bootstrap confidence intervals where practical, especially for small tail classes.
- Do not infer successful contribution preservation from macro-F1 alone; report predictive and contribution metrics together.

13 Reproducibility and implementation checklist

Suggested project structure

```
project/
  configs/              dataset, model, targets, training
  data/                 manifests, cached_features
  src/
    data_adapters/
    model/              encoders, attention, fusion, classifier,
   missing_modality
    contribution/       risk_estimator, fcd_estimator, target_builder,
   signatures
    grouping/           cluster_profiles, fallbacks
    losses/             class_balanced, group_balanced, alignment
    optim/              fixed_weight, moo
  train_reference.py
  train_joint.py
  evaluate.py
  outputs/              checkpoints, logs, tables, plots
```

Configuration fields to save

Area	Required settings
Data	Manifest, label map, modality paths/features, source ID, split, temporal alignment.
Model	Encoder IDs/versions, projection dimension, attention heads, fusion, mask/token, pooling.
Reference training	Seed, batch size, optimizer, learning rate, weight decay, epochs, early-stop metric.
Contribution	UCD/FCD estimators, ablation path, estimation folds, ε, bootstrap count.
Grouping	Scaler, linkage, candidate group counts, minimum support, fallback rule.
Optimization	CI objective, target granularity, λ grid or MOO algorithm, trainable parameters.
Evaluation	Metrics, seeds, checkpoint rule, long-tail ratios, per-class report.

Required saved artifacts

- Reference and final checkpoints plus exact encoder identifiers.
- Data manifest, split IDs, label mapping, controlled long-tail IDs and seeds.
- Signed sample signatures, UCD targets, FCD estimates, scaler parameters and group assignments.
- Group counts, cluster diagnostics, support/fallback logs and uncertainty summaries.
- Training logs, objective values, gradient-conflict diagnostics, configuration files and environment versions.
- Per-seed predictions, metrics, figures and scripts that regenerate the result tables.

Resource starting point

On a 6 GB GPU, begin with frozen encoders and cached features; try batch size 4–16 depending on feature shape, mixed precision if stable, and one training process at a time. Profile memory before increasing batch size or unfreezing encoders.

Completion checklist

- ☐ Data alignment and source-independent split verified.
- ☐ Baseline and missing-modality pathway validated.
- ☐ UCD/FCD estimators unit-tested and documented.
- ☐ Target fallback and grouping support rules fixed before test evaluation.
- ☐ Class-balanced and group-balanced losses unit-tested.
- ☐ Fixed-weight and MOO updates use the same trainable parameter set.
- ☐ All target granularities evaluated with controlled ablations.
- ☐ Predictive metrics and contribution metrics reported together.

14 References and resources

Method references

[1] **Utopia** — "Dataset-aware Utopia modality contribution for imbalanced multimodal learning," *Information Fusion* (2025). Use the original paper for the exact UCD/FCD estimators and implementation details; this document describes a Utopia-inspired target-alignment framework and explicitly labels its simplified risk estimator as an instantiation.

[2] Hou et al., "Subclass-balancing Contrastive Learning for Long-tailed Recognition," *ICCV* 2023. Used as inspiration for within-class structure; the core method here does not include the original SBCL contrastive objective.

[3] "Long-Tailed Learning as Multi-Objective Optimization," *AAAI* 2024. Background for treating class-level risks as competing objectives.

Dataset resources

- RAVDESS — official Zenodo record
- AVE — official project repository
- CMU-MOSI — CMU MultiComp page
- CMU-MOSEI — CMU MultiComp page
- CMU Multimodal SDK — official GitHub

Final implementation note. Before publishing, verify the exact UCD/FCD equations against the Utopia paper and the chosen MOO update against its original algorithm or reference implementation. The risk-based UCD formula included here is a transparent, reproducible starting estimator; it should not be described as an exact reproduction of Utopia unless verified. The same applies to contribution-profile grouping: it is a proposed adaptation, not the original SBCL method.